Theoretical Guide
py.Sandu

Élvis Miranda, Leonardo Krauss & Rafael Alencar

1 Counting Problems

2 Notes

3 Identities

4 C++

4.1 System optimization

For faster IO, we can use the following code:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
cout.tie(nullptr);
```

This code disables the synchronization between C++ streams and C streams, which can significantly improve the performance of input and output operations. Additionally, it unties the input and output streams, allowing them to operate independently, further enhancing the speed of IO operations.

4.2 Containers

- **std::vector:** A dynamic array. Most methods give O(1) time complexity for access and O(N) for insertion/deletion (except at the end, which is O(1)).
- **std::list:** A doubly linked list. Most methods give O(1) time complexity for insertion/deletion (given an iterator) and O(N) for access.
- **std::deque:** A double-ended queue. Most methods give O(1) time complexity for access and O(1) for insertion/deletion at both ends, but O(N) for insertion/deletion in the middle.
- **std::stack:** A container adapter that gives the functionality of a stack (LIFO). Most methods give O(1) time complexity for push, pop, and top operations.
- **std::queue:** A container adapter that gives the functionality of a queue (FIFO). Most methods give O(1) time complexity for push, pop, and front/back operations.
- **std::priority_queue:** A container adapter that gives the functionality of a priority queue. Most methods give O(log N) time complexity for push and pop operations, and O(1) for top operation.
- **std::set:** A sorted associative container that contains unique elements. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::map:** A sorted associative container that contains key-value pairs with unique keys. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::unordered_set:** An unordered associative container that contains unique elements. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.
- **std::unordered_map:** An unordered associative container that contains key-value pairs with unique keys. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.

4.3 Algorithms

- **std::sort:** The `std::sort` function in C++ is typically implemented using a hybrid sorting algorithm called Introsort, which combines Quick Sort, Heap Sort, and Insertion Sort. It has an average time complexity of O(N log N).
- **std::lower_bound and std::upper_bound:** These functions are used to find the lower and upper bounds of a value in a sorted range. They have a time complexity of O(log N) since they use binary search.
- **std::next_permutation:** This function generates the next lexicographical permutation of a sequence. It is useful for generating all permutations

of a sequence in sorted order. The time complexity is O(N) for each call, where N is the length of the sequence.

5 Math

6 Constants

6.1 Time and memory limits

- **1 second limit:** 1 second is usually enough for a program to perform around 10^8 operations for O(N) algorithms and around 10^6 operations for O(N log N) algorithms.
- **256Mb limit:** 256 megabytes is usually enough to store around 10^7 integers (since each integer typically takes 4 bytes).

6.2 Mathematical

- **Pi (π)**: Usually defined as acos(-1.0), approximately equal to 3.14159. In C++ 20, it is available as `std::numbers::pi`.
- **Euler (e)**: Usually defined as the base of the natural logarithm, approximately equal to 2.71828. In C++ 20, it is available as `std::numbers::e`.
- **Golden Ratio (ϕ)**: Usually defined as (1 + sqrt(5)) / 2, approximately equal to 1.61803. In C++ 20, it is available as `std::numbers::phi`.

6.3 Data type limits

- **Integer limits**: The typical 32 bit signed integer bounds are ($\approx \pm 2 \times 10^9$) and is obtained by using the `INT_MAX` and `INT_MIN` constants, while the 64 bit signed integer bounds are ($\approx \pm 9 \times 10^{18}$) and is obtained by using the `LLONG_MAX` and `LLONG_MIN` constants.
- **Floating-point limits**: The typical 32 bit floating-point bounds are approximately $\pm 3.4 \times 10^{38}$ and is obtained by using the `FLT_MAX` and `FLT_MIN` constants, while the 64 bit floating-point bounds are approximately $\pm 1.7 \times 10^{308}$ and is obtained by using the `DBL_MAX` and `DBL_MIN` constants.

7 Number Theory

7.1 Modular Arithmetic & Number Theory

- **Properties of Modulo:**
  - Sum: $(a+b)\%m = (a\%m + b\%m)\%m$
  - Subtraction: $(a-b)\%m = (a\%m - b\%m + m)\%m$
  - Product: $(a \times b)\%m = (a\%m \times b\%m)\%m$
- **Modular Division:** $(a/b) \equiv (a \times b^{-1}) \pmod m$, where $b \times b^{-1} \equiv 1 \pmod m$. Valid if $\gcd(b, m) = 1$.
- **Euler's Totient Theorem:** If $\gcd(a, m) = 1$, then $a^{\phi(m)} \equiv 1 \pmod m$.
- **Fermat's Little Theorem:** If p is prime, $a^{p-1} \equiv 1 \pmod p$. Inverse: $a^{-1} \equiv a^{p-2} \pmod p$.
- **Power Reduction:** If p is prime: $a^b \equiv a^{b \pmod{p-1}} \pmod p$. General: $a^b \equiv a^{b \pmod{\phi(m)}} \pmod m$ (if $\gcd(a, m) = 1$).
- **Bézout's Identity:** For a, b, there exist x, y such that $ax + by = \gcd(a, b)$.

7.2 Divisors and Prime Factorization

For an integer $n > 1$, let its prime factorization be $n = p_1^{a_1} \cdot p_2^{a_2} \dots p_k^{a_k}$.

- **Number of Divisors (τ):** The total count of divisors of n.

$$\tau(n) = \prod_{i=1}^{k} (a_i + 1)$$

- **Sum of Divisors (σ):** The sum of all positive divisors of n.

$$\sigma(n) = \prod_{i=1}^{k} \left(\frac{p_i^{a_i+1} - 1}{p_i - 1} \right)$$

- **Product of Divisors (P):** The product of all divisors of n.

$$P(n) = n^{\frac{\tau(n)}{2}}$$

Note: If n is a perfect square, $\tau(n)$ is odd, but the result remains an integer.

- **Euler's Totient Function (ϕ):** Number of integers $k \in [1, n]$ such that $\gcd(k, n) = 1$.

$$\phi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right)$$

8 Bitwise

9 Geometry

9.1 Lines & Polygons

- **Slope-Intercept:** $y = mx + c$
- **General Line Equation:** $Ax + By + C = 0$
- **Distance from point (x_0, y_0) to line $Ax + By + C = 0$:** $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$
- **Shoelace Formula (Area of Polygon):** $A = \frac{1}{2} |\sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n)|$
  OBS: All vertices must be in order, either clockwise or counterclockwise
- **Pick's Theorem (Area of Lattice Polygons):** $A = I + \frac{B}{2} - 1$, where I is interior points and B is boundary points

9.2 Circles

- **Area:** πr^2
- **Circumference:** $2\pi r$
- **Equation:** $(x - h)^2 + (y - k)^2 = r^2$

9.3 Basic 2D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2}$
- **Vector Definition:** $\vec{v} = (x, y)$
- **Dot Product $(\vec{a} \cdot \vec{b})$:** $x_1 x_2 + y_1 y_2 = |\vec{a}||\vec{b}| \cos\theta$
  Usage: Finding angles, checking perpendicularity $(\vec{a} \cdot \vec{b} = 0)$
- **Cross Product $(\vec{a} \times \vec{b})$:** $x_1 y_2 - y_1 x_2 = |\vec{a}||\vec{b}| \sin\theta$
  Usage: 2D orientatiom, calculating signed area, collinearity $(\vec{a} \times \vec{b} = 0)$
- **Midpoint:** $M = (\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$

9.4 Algorithms/Concepts

- **Convex Hull (Monotone Chain or Graham Scan):** Finds the smallest convex polygon containging all points
- **Line Segment Intersection:** Using cross products to check if two segments intersect
- **Point in Polygon:** Using ray casting (winding number) algorithm for O(n) complexity or BSP Tree for O(log n) complexity
- **Angle of vector:** Use `atan2(y,x)` to find the angle of a vector.

10 Progressions

- **Arithmetic Progression (AP):** Sum of the first n terms:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Sum of Squares:** Sum of the first n squares:

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

- **Sum of Cubes:** Sum of the first n cubes:

$$\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \left(\frac{n(n+1)}{2}\right)^2$$

- **Geometric Progression (GP):** Sum of the first n terms (ratio $r \neq 1$):

$$\sum_{i=0}^{n-1} a \cdot r^i = a\left(\frac{r^n - 1}{r - 1}\right)$$

Note: If $|r| < 1$, the infinite sum is $S_\infty = \frac{a}{1-r}$.

- **Arithmetico-Geometric Series:** Sum of $\sum_{i=1}^{n} i \cdot r^{i-1}$ (common in expected value problems):

$$\sum_{i=1}^{n} i \cdot r^{i-1} = \frac{1 - r^n}{(1-r)^2} - \frac{nr^n}{1-r}$$